

XRPL Validator — Daily Engineering Report • 2026-09-10

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **9 commits** (xrpl-validator 9)
- **9 engine findings fixed** (F243–F251)
- Amendment **fixCleanup3_3_0** held majority (since 2026-08-28 11:15 UTC); activation due ~2026-09-11 11:15 UTC

Engine findings fixed

- **fix(xrpl-ledger): F247 — a two-asset deposit that rounds to zero LP tokens is tecAMM_INVALID_TOKENS** (a3aa092 , xrpl-validator) equalDepositLimit (AMMDeposit.cpp:735-741, :762-767) reports a lead direction whose tokensAdj == beast::zero as tecAMM_INVALID_TOKENS under fixAMMv1_3, judged before that direction's fit and ending the search; only "neither direction fits" is tecAMM_FAILED. Our equal_deposit_limit folded both into one None → tecAMM_FAILED. #106894782 ED009182D8E8: 7e-13 589 and 6e-13 PLX into the 589/PLX pool — fee-only either ...
- **fix(xrpl-ledger): F246 — anyone may cancel an EXPIRED check** (48567ac , xrpl-validator) CancelCheck.cpp:56-66 restricts the cancel to the writer or the destination only while the check is live: if (!optExpiry || parentCloseTime < Expiration) guards the tecNO_PERMISSION test, so an expired check is cancellable by any account. We tested the parties unconditionally. #106900377 A252917DB8E8 (then #106900393, #106900408): rpcjmdWFCQ — neither writer nor destination — cancels rDoxEVqFy3's check 3FF6163A to ...
- **fix(xrpl-ledger): F245 — accepting an expired NFTokenOffer deletes it and is tecEXPIRED** (c38fa03 , xrpl-validator) rippled 3.3.0 under fixCleanup3_1_3 (NFTokenAcceptOffer.cpp:69-75, :446-477): an expired offer passes preclaim, and doApply's FIRST act deletes every named offer that has expired — hasExpired(view(), Expiration), i.e. parentCloseTime >= Expiration, buy then sell — and returns tecEXPIRED if any was. The Transactor's processPersistentChanges then re-applies exactly those erasures after the tec reset (ltNFTOKEN_OFFER ...
- **fix(xrpl-ledger): F244 — a bridged crossing's leg-B rev extent is withdrawn: it reaped offers rippled's stream never steps onto** (60b4142 , xrpl-validator) Finding 239 added rev_extent_reap to cross_bridged so a bridged strand's rev pass would remove expired/unfunded offers past the level it consumed whole (#106863376). Four ledgers of the deploy109 soak show its reach is wrong: #106873753 C4FEF57142AA — rMsXVzCug7's tfPassive BTC → RLUSD offer — crosses NOTHING (mainnet touches six nodes and rests it), yet the extent walked the BTC → XRP → RLUSD bridge and reaped ...
- **fix(xrpl-ledger): F248 — a payment hop's limiting book pass re-runs with the level's sixteen-digit fold, like a crossing's** (620104b , xrpl-validator) Finding 149 ported StrandFlow's limiting re-execution for OfferCreate

crossings only (`fold_rem`): when a step's rev output is not what the strand asked for, rippled throws the pass away and re-runs that step's rev with its own 16-digit total as the target (StrandFlow.h:176-184), so the level's LAST whole offer is re-sized by `limitStepOut` to `total16 - Σprev` at the page rate. The rule is the strand's, not the ...

- **fix(xrpl-ledger): F249 — a bridge member consumed at its own funds cap rounds its in DOWN** (`1613c3b`, `xrpl-validator`) rippled's `forEachOffer` funds branch (BookStep.cpp:779-793): when the owner cannot fund the whole offer, `ownerGives = funds; stpAmt.out = mulRatio(funds, ..., roundUp = false); ofrAmt = offer.limitOut(ofrAmt, stpAmt.out, roundUp = false)` — the member's in is the page-rate product rounded DOWN ("so that the quality of an offer left in the ledger is as good or better than its book page"). Only a slice the STRAND cuts ...
- **fix(xrpl-ledger): F250 — a bridge level's ending member keeps its own ceil_out; the fold's lost digit is the taker's** (`c7ce688`, `xrpl-validator`) In an OUT-limited pass rippled sizes the level's ending member with `limitStepOut` — `offer.limitOut(ofrAmt, out', roundUp = true)`, i.e. its OWN `ceil_out` at the page rate — and the pass's in is `sum(savedIns)`, the sixteen-digit fold of the members' ins (BookStep.cpp `revImp`). When that fold drops a seventeenth digit, the taker pays the folded figure and the makers keep theirs. Our dealing plan gave the ending member ...
- **fix(xrpl-ledger): F243 — a crossing's pool round is capped by the GROSS fold remainder, not the net remainder re-grossed** (`6328e1b`, `xrpl-validator`) rippled's strand keeps `remainingIn` gross-primary — `maxIn - sum(savedIns)` at sixteen digits (StrandFlow) — and an iteration that exhausts it debits the taker that remainder verbatim; the net the pool receives is derived from it. `cross_bridged` handed its pool turns (`consume / consume_fib`) a round cap only when the trust line bound tighter (finding 212); otherwise the exhausting slice re-grossed the NET remainder ...
- **fix(xrpl-ledger): F251 — a bridged leg's rev extent reaps only after a CLOB-served pass; a pool-served pass never steps onto the book** (`5de9f43`, `xrpl-validator`) Finding 239 gave leg B of a bridged crossing rippled's rev-pass reach — after consuming a level, the stream steps on, removing the expired and unfunded offers it lands on until the first live one. Finding 244 withdrew it: on `#106873753 C4FEF57142AA` (`rMsXVzCug7`'s `tfPassive` BTC → RLUSD offer, which crosses nothing) the extent removed `rLDyWWMiW6`'s dead `EE68DEE8` and its pages, four ledgers running, while mainnet left ...

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Amendment	Majority since	Activation due
fixCleanup3_3_0	2026-08-28 11:15 UTC	2026-09-11 11:15 UTC

Amendments enabled on mainnet: 93.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **State-Triggered Escrow via Credential Status** (payments) Add a `tfStateConditional` flag to `EscrowCreate` plus SFields `sfConditionKeylet` (Hash256) and `sfConditionCode` (UInt8 enum: `CREDENTIAL_VALID`, `CREDENTIAL_R`)
- **Oracle-Bounded Slippage Protection for IOC Offers** (dex) Add a `tfProtectedFill` flag to `OfferCreate` plus SFields `sfPriceOracleID` (a `PriceOracle` keylet) and `sfMaxSlippageBps` (UInt16). In the existing offer-cro
- **Validator Fee-Vote Extended to Tunable Parameters** (governance) Add a new `ltPARAMS` singleton ledger object holding a small fixed whitelist of tunable numeric constants (initially: fee-escalation multiplier, tx-queue

Appendix — commits

Time	Repo	Commit	Subject
22:55	xrpl-validator	a3aa092	fix(xrpl-ledger): F247 — a two-asset deposit that rounds to zero LP tokens is <code>tecAMM_INVALID_TOKENS</code>
22:55	xrpl-validator	48567ac	fix(xrpl-ledger): F246 — anyone may cancel an <code>EXPIRED</code> check
22:55	xrpl-validator	c38fa03	fix(xrpl-ledger): F245 — accepting an expired <code>NFTokenOffer</code> deletes it and is <code>tecEXPIRED</code>
22:55	xrpl-validator	60b4142	fix(xrpl-ledger): F244 — a bridged crossing's leg-B rev extent is withdrawn: it reaped offers rippled's stream never steps onto
23:15	xrpl-validator	620104b	fix(xrpl-ledger): F248 — a payment hop's limiting book pass re-runs with the level's sixteen-digit fold, like a crossing's
23:19	xrpl-validator	1613c3b	fix(xrpl-ledger): F249 — a bridge member consumed at its own funds cap rounds its in <code>DOWN</code>
23:31	xrpl-validator	c7ce688	fix(xrpl-ledger): F250 — a bridge level's ending member keeps its own <code>ceil_out</code> ; the fold's lost digit is the taker's
23:43	xrpl-validator	6328e1b	fix(xrpl-ledger): F243 — a crossing's pool round is capped by the <code>GROSS</code> fold remainder, not the net remainder re-grossed
23:59	xrpl-validator	5de9f43	fix(xrpl-ledger): F251 — a bridged leg's rev extent reaps only after a <code>CLOB</code> -served pass; a pool-served pass never steps onto the book